

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

https://github.com/SamrithaShree/Deep_Learning/tree/main/Lab-3

10. Generated Plots with Interpretation

1. Sample Images

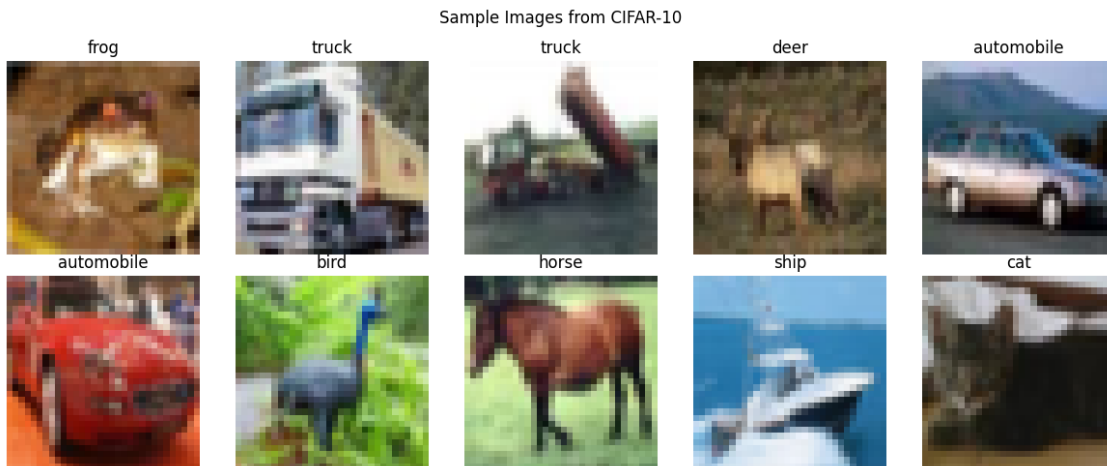


Figure 1: Ten Sample Images from CIFAR-10 with Class Labels

Purpose: Visual Sanity Check

Interpretation: This confirms the correct dataset was loaded. However the data seem to be small and blurred due to the 32x32 resolution.

2. Class Distribution

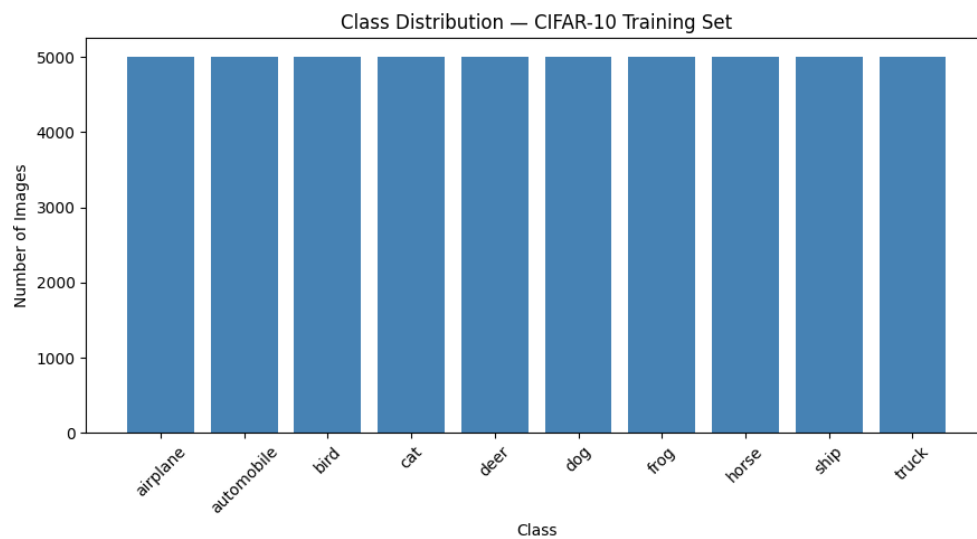


Figure 2: Class Distribution of the CIFAR-10 Training Set

Purpose: Balance Check

Interpretation: This shows the dataset is perfectly balanced dataset.

3. Training Vs Validation Accuracy

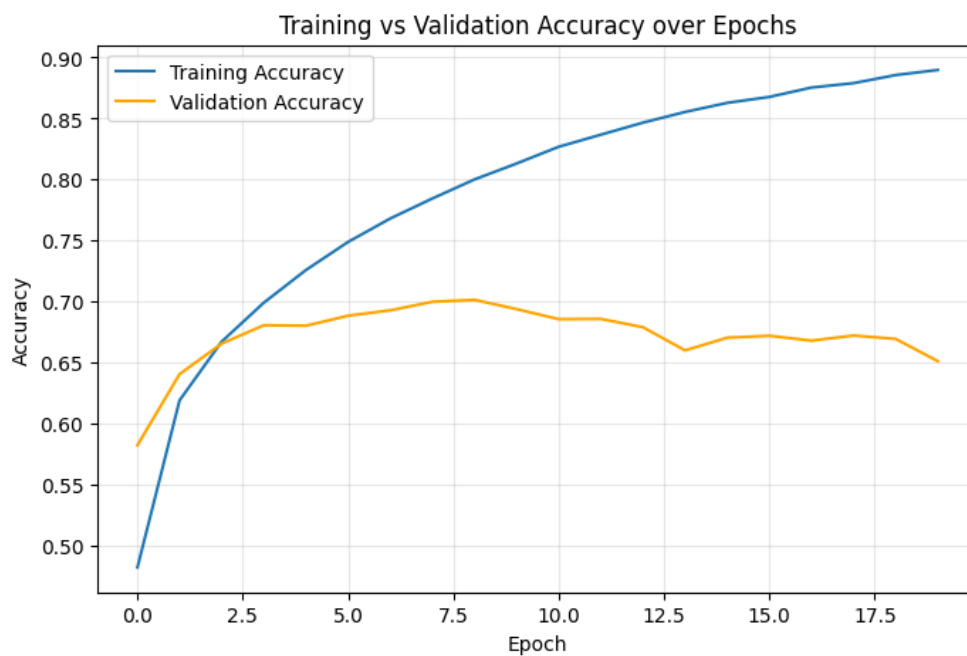


Figure 3: Training Accuracy over 20 Epochs

Purpose: Learning Behaviour

Interpretation: Training accuracy is constantly increasing until the last epoch denoting its learning well. But, the validation accuracy is after a very short time starting to decrease and increase but lower than the training accuracy. This shows overfitting.

4. Training Vs Validation Loss

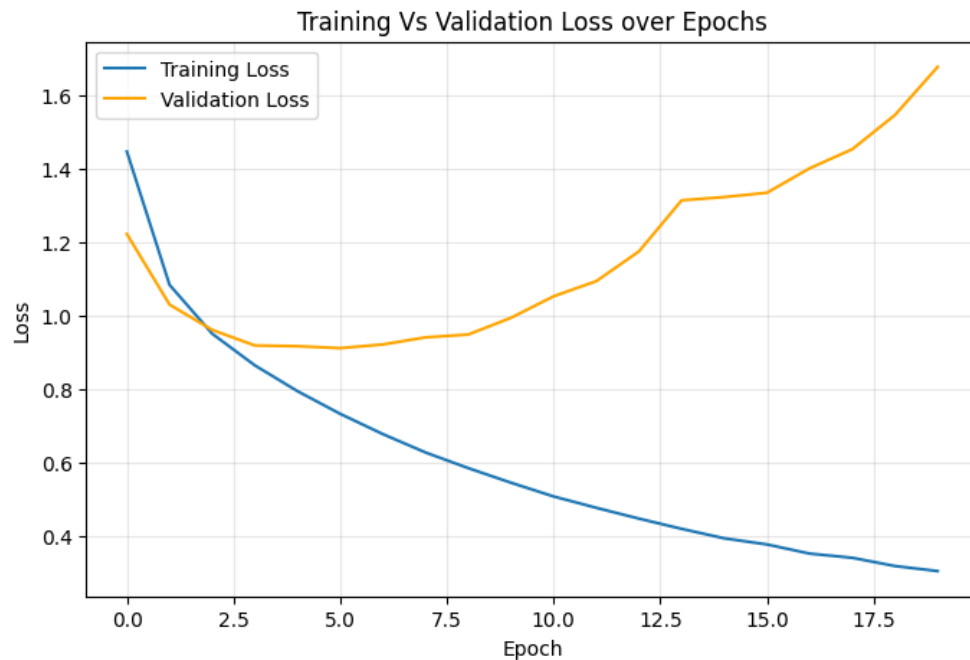


Figure 4: Validation Accuracy over 20 Epochs

Purpose: Generalization Check

Interpretation: Learning is extremely good in training that the training loss is continuously reducing. Whereas the validation loss after some 4 epochs is starting to increase which shows overfitting. The model is unable to generalize well.

7. Feature Maps

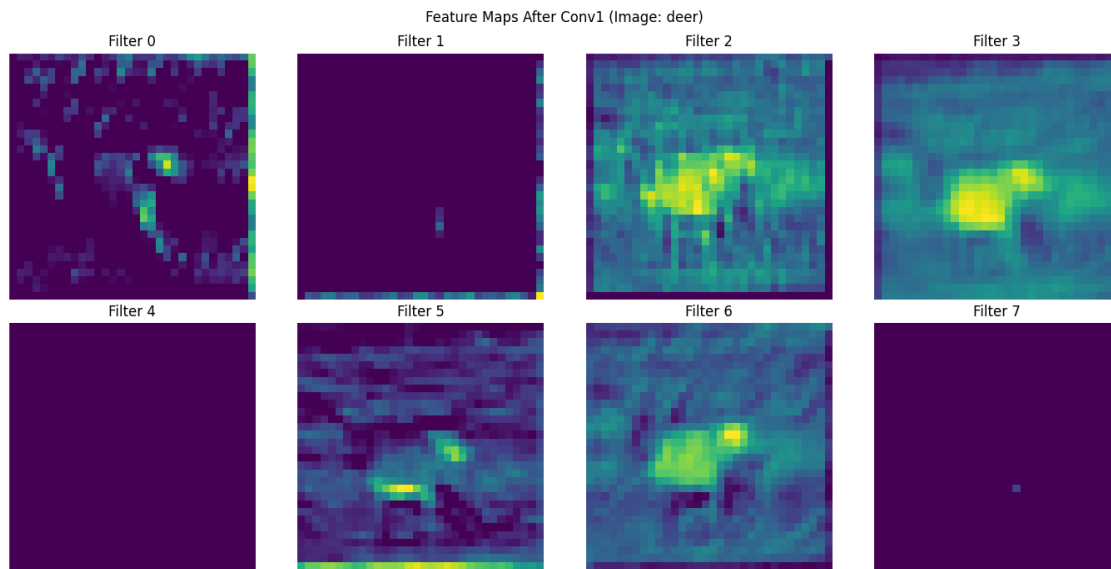


Figure 5: Feature Maps after the First Convolution Layer

Purpose: Feature Extraction Check

Interpretation: Different filters clearly respond to different aspects of the same input image – some highlight edges and outlines, some react to specific colour regions, and a few appear almost blank.

8. Confusion Matrix

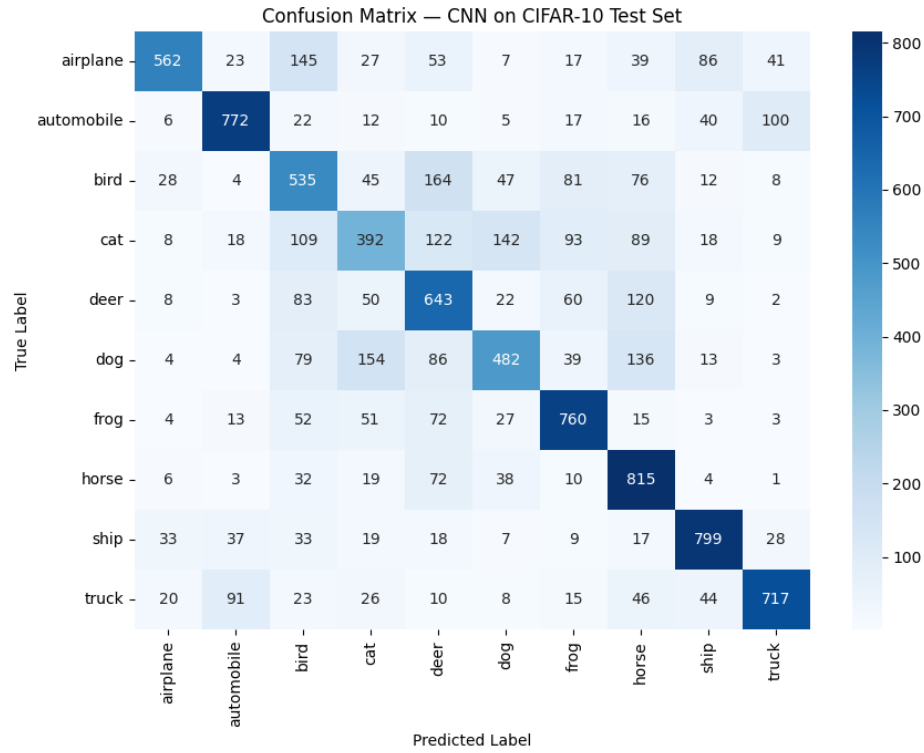


Figure 6: Confusion Matrix on the CIFAR-10 Test Set

Purpose: Per-Class Performance Evaluation

Interpretation: The confusion matrix is showing decent classification behaviour of the model. It is noticeable that the closely related classes are mostly misclassified among all.

11. Additional Exercises

1. **Output size for a 64×64 image, 5×5 kernel, stride 2, padding 2**

$$\text{Output} = \frac{64 - 5 + 2(2)}{2} + 1 = \frac{63}{2} + 1 = 31.5 + 1$$

Since the number of output positions must be an integer, this is floored: $\lfloor 31.5 \rfloor + 1 = 32$.

Output size = 32×32

2. **Trainable parameters for 64 filters of size 3×3 , RGB input**

$$\text{Params} = (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1792$$

Trainable parameters = 1792

3. **ReLU vs Sigmoid Activation Functions**

Aspect	ReLU	Sigmoid
Formula	$f(z) = \max(0, z)$	$f(z) = \frac{1}{1 + e^{-z}}$
Output Range	$[0, \infty)$	$(0, 1)$
Vanishing Gradient	Mostly avoided for $z > 0$	Severe for large $ z $
Computation Cost	Very cheap	Costlier (exponential)
Typical Use	Hidden layers in CNNs	Binary output layers

4. Max Pooling vs Average Pooling – Results

Pooling Type	Test Accuracy
Max Pooling	64.77%
Average Pooling	64.41%

Max pooling gave a small but consistent accuracy improvement over average pooling under identical training settings. This lines up with the intuition that max pooling preserves the strongest, most discriminative activation in each window, while average pooling dilutes it by blending in weaker neighbouring activations – which tends to matter more for image data where sharp edges and shapes carry most of the class information.

5. Effect of Increasing Filters from 16 to 64

Configuration	Params (Conv1)	Training Time (2 epochs)
16 filters	448	15.9 s
64 filters	1792	19.3 s

Increasing the number of filters gives the network more capacity to learn a wider variety of features per layer, which generally improves accuracy – but with clearly diminishing returns beyond a point. The trade-off is real: more filters means more parameters, more computation per forward/backward pass, and noticeably longer training time, so the choice comes down to balancing accuracy gains against the available compute budget.

12. Results

Training Accuracy	88.98%
Testing Accuracy	64.77%
Precision	65.76%
Recall	64.77%
F1-score	64.58%
Number of Parameters	282,250

13. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution layers are computationally efficient than fully connected layers due to their less no of parameters. This is because convolution layers shares weights whereas fully connected layers use unique weights for every neuron. Convolution layers also retain the spatial topology as well as it is translation invariant.

2. How does stride affect the feature map size?

Stride has a direct impact on the feature map size. As stride increases, the feature map size decreases because the stride is the no of pixels the kernel of the filter moves or skips in each steps. So as it skips more the size reduces.

3. What is the role of padding?

Padding helps to maintain the feature map size especially in case of small images so that it doesn't shrink faster and allows the model for deep learning.

4. Why is pooling used?

Pooling helps the cnn in maintaing translation invariance, increasing the local receptive field and also ignores noise and reduces overfitting.

5. How do feature maps represent image characteristics?

Feature maps are the outputs of each learned filters across the image. Each feature maps shows light up pixels for what -attern each filter is trying to learn. the deeper filters learn more complex patterns.

6. Why do CNNs require fewer parameters than MLPs?

CNN requires fewer parameters then MLP because they share weights and the no of weights does not depend on the input size whereas in an MLP, it directly depends on the input size and every single neuron has unique weights and requires more parameters.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.